

Github action

ONE LOVE. ONE FUTURE.

1

1

Automating Testing and Safe Code Merging

- Testing code locally can be cumbersome due to:
 - The need for frequent runs to catch bugs early
 - Extensive tests for high code coverage, which may take time
- Automating tests for each repository push can help
- Merging branches only after passing automated tests ensures safer code, assuming well-covered tests



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

2

2

Using GitHub Actions for Continuous Integration

- GitHub Actions offers a continuous integration solution.
- Each repository receives 2,000 minutes of free testing per month, sufficient for course and personal projects.
- Initial setup of GitHub Actions may seem complicated.
- Workflow files created for one repository can be reused for other repositories.



3

Organization of a GitHub Workflow File

- Start by giving the workflow a **name**.
- Specify **trigger events** for the workflow:
 - Include actions (e.g., pull request, push).
 - Define applicable branches.
- List the **jobs** to be executed:
 - Jobs run in parallel by default but can have dependencies.
- Use **runs-on** to specify the operating system for the workflow.
- Define the **steps**, where actual commands to be run are specified.



4

```

name: testing
on:
  pull_request:
    branches:
      - main
jobs:
  test-code:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repo
        uses: actions/checkout@v2
      - name: Install dependencies
        run: make install-test
      - name: Execute tests
        run: pytest tests/tagifai

```

- ◀ Event (single or multiple)

schedule: * * * * *
manual
on: push

on: pull_request
on: issues
...
- ◀ Jobs (In parallel or dependent)
- ◀ Runner
- ◀ Steps
- ◀ Actions (from Marketplace, custom, etc.)
- ◀ Commands

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

5

5

Exercises

1. Create a **.github** folder in the root of your repository, and add a sub-folder called **workflows**.
2. Review the [GitHub Actions guide for Python](#) (focusing on automated testing for Python code)
3. Place the provided workflow file called **tests.yaml** in the **.github/workflows/** folder.
4. Defining requirements.txt and requirements_tests.txt
5. Pushing Changes to Repository and Running Tests
6. Expanding CI Tests for Multiple OS and Python Versions
7. Implementing Caching in Workflow
8. Adding Code Coverage to Workflow
9. Setting Up Branch Protection Rules
10. Continuous Integration Data Testing

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

6

6

Workflow File

- Place the provided workflow file called **tests.yaml** in the **.github/workflows/** folder. This workflow file includes three steps:
 - **Initiate a Python environment** (Python 3.8 in this case).
 - **Install all dependencies** required to run the tests.
 - **Call `pytest`** to execute the tests.
- Review the file to understand its overall structure and syntax.



7

Example of tests.yaml

```
name: "Run tests"
on:
  push:
    branches: [ master, main ]
  pull_request:
    branches: [ master, main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v2
      - name: Set up Python 3.8
        uses: actions/setup-python@v2
        with:
          python-version: 3.8
      - name: Install dependencies
        run:
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install -r requirements_tests.txt
      - name: Test with pytest
        run:
          pytest -v
```



8

4. Defining `requirements.txt` and `requirements_tests.txt`

- `requirements.txt`:
 - Contains all packages necessary to run the code.
- `requirements_tests.txt`:
 - Contains additional packages required for running tests.
 - Can be empty if no extra testing packages are needed.
 - Used when specific testing libraries are not needed for the main code to function.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

9

9

5. Pushing Changes to Repository and Running Tests

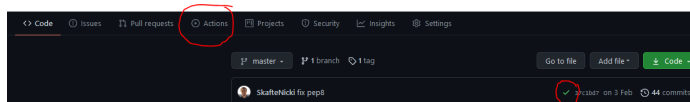
Push changes to the repository.

Tests should automatically start after pushing.

Expect a green check mark next to the commit hash once tests pass.

Inspect the **Actions** tab:

View the history of actions run.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

10

10

6. Expanding CI Tests for Multiple OS and Python Versions

1. The provided tests .yaml runs on **one operating system**:

- Likely ubuntu-latest.

2. To execute tests on two other main operating systems:

- Use the strategy attribute to define a matrix for multiple OS.

```
jobs:
  build:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: ["ubuntu-latest", "windows-latest", "macos-
latest"]
```



6. Expanding CI Tests for Multiple OS and Python Versions (2)

3. To run the tests on different Python versions:

- Add a python-version matrix.

```
jobs:
  build:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: ["ubuntu-latest", "windows-latest", "macos-
latest"]
        python-version: [3.10, 3.11, 3.12]
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
```



7. Implementing Caching in Workflow

- GitHub actions will destroy every downloaded package when the workflow has been executed
- 1. To implement caching in your workflow file, follow the steps below:
 - Use the cache option within the setup-python action to cache dependencies.

```
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-python@v5
    with:
      python-version: '3.10'
      cache: 'pip' # Caching pip dependencies
  - run: pip install -r requirements.txt
```



7. Implementing Caching in Workflow (2)

2. After implementing the caching system:
 - Go to **Actions** -> **Caches** in your repository to confirm the cache has been added.
 - It should display the cached dependencies as shown in the image below:
3. Measure workflow runtime before and after adding caching:
 - Compare times to see if caching improves the workflow's runtime.



8. Adding Code Coverage to Workflow

To add code coverage to your workflow, follow the steps outlined in the linked [post](#):

1. Install coverage and codecov in your environment by adding them to `requirements.txt` or installing them directly in your GitHub Actions workflow.

```
pip install coverage codecov
```



8. Adding Code Coverage to Workflow

2. Update your workflow file to include steps for running coverage and uploading results:

```
steps:
- uses: actions/checkout@v4
- uses: actions/setup-python@v5
  with:
    python-version: '3.10'
- run: pip install -r requirements.txt
- run: coverage run -m pytest # Run tests with coverage
- run: coverage xml # Generate coverage report in XML format
- name: Upload coverage to Codecov
  uses: codecov/codecov-action@v3
  with:
    file: ./coverage.xml # Upload the coverage report
```

3. Ensure your repository is set up with [Codecov](#) by logging in and adding your project.
4. Once implemented, your workflow will upload the code coverage report as an artifact after running the tests.



9. Setting Up Branch Protection Rules

1. Create a Branch Ruleset:
2. Setting Rules for Branch Protection
3. Implementing Branch Protection Rules with Admin Bypass

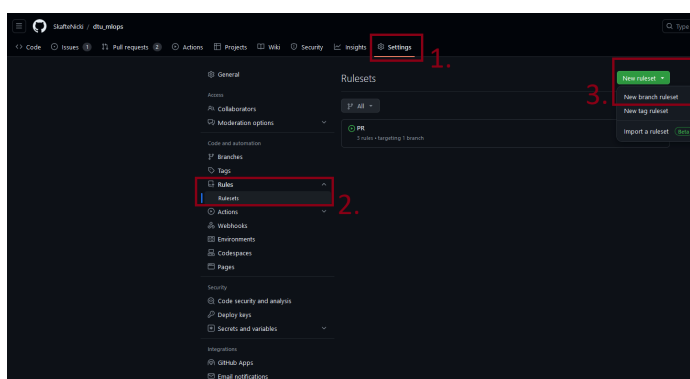


17

Create a Branch Ruleset:

Go to **Settings** -> **Rules** -> **Rulesets** in your repository.

Create a new branch ruleset to define conditions that must be met before code is merged into the main branch.



18

Setting Rules for Branch Protection

1. Name the ruleset and set the target branches to the **Default branch** (e.g., main or master).
2. Key rules to consider:
 - **Require a pull request before merging:**
 - Ensures changes are merged via pull requests.
 - Allows for code review and testing before merging into the main branch.
 - Optionally, set a requirement for a specific number of reviewers or approvals.
 - **Require status checks to pass:**
 - Ensures all workflows pass before merging.
 - Select which workflows are mandatory (e.g., tests, code coverage).



☒ **Require a pull request before merging**
Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.

Hide additional settings ^

Required approvals
2

The number of approving reviews that are required before a pull request can be merged.

☐ **Dismiss stale pull request approvals when new commits are pushed**
New reviewable commits pushed will dismiss previous pull request review approvals.

☐ **Require review from Code Owners**
Require an approving review in pull requests that modify files that have a designated code owner.

☐ **Require approval of the most recent reviewable push**
Whether the most recent reviewable push must be approved by someone other than the person who pushed it.

☐ **Require conversation resolution before merging**
All conversations on code must be resolved before a pull request can be merged.

☒ **Require status checks to pass**
Choose which status checks must pass before the ref is updated. When enabled, commits must first be pushed to another ref where the checks pass.

Hide additional settings ^

☒ **Require branches to be up to date before merging**
Whether pull requests targeting a matching branch must be tested with the latest code. This setting will not take effect unless at least one status check is enabled.

Status checks that are required + Add checks

Check Markdown formatting	GitHub Actions	
Check Markdown links	GitHub Actions	
Check code formatting	GitHub Actions	



Implementing Branch Protection Rules with Admin Bypass

1. Required rules to implement:

- **At least one person** needs to approve any pull request.
- **All workflows** must pass before merging.
- **All conversations** must be resolved before merging.

2. Admin Bypass:

- If the rules feel too restrictive, allow the **repository admin** to bypass the rules.
- Add the **Repository admin** to the bypass list in the ruleset.

This allows for flexibility while ensuring necessary checks and approvals are in place before code is merged into the main branch.

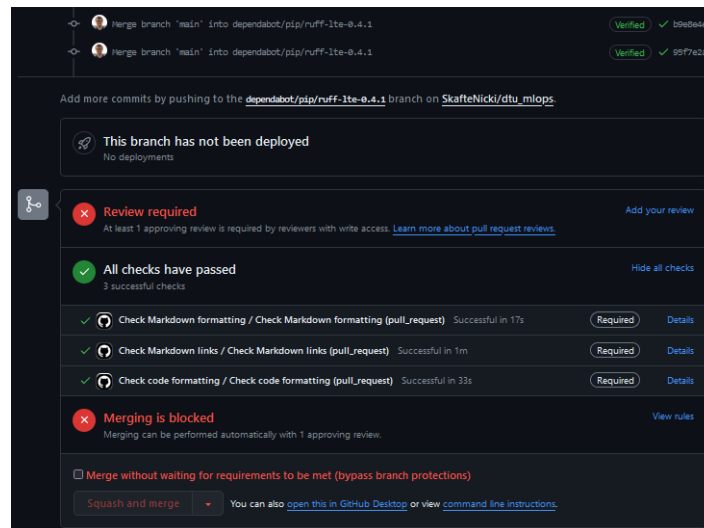


3. Implementing Branch Protection Rules with Admin Bypass (2)

3. Successful Implementation:

- If the rules are created correctly, you should see a notification similar to the image below when attempting to merge a pull request.
- All three checks must pass before merging:
 - At least one approval from a reviewer.
 - All workflows must be successful.
 - All conversations must be resolved.
- A bypass rule for the repository admin should also be visible, allowing them to merge regardless of the checks.





23

10. Continuous Integration Data Testing

- Problem encountered: inability to run tests due to data not stored in GitHub.
- Assumption: Module M8 - DVC has been completed.
- Core issue: Data needs to be available for testing.
- Solution proposed: Download data during continuous integration (CI) to enable testing.



24

10.1 CI Pipeline Authentication Issue

- Problem identified: CI pipeline needs to authenticate with the storage solution.
- Solution: Use an authentication file generated during the first DVC push.
- File location:
 - `$CACHE_HOME/pydrive2fs/{gdrive_client_id}/default.json`
 - macOS: `~/Library/Caches`
 - Linux: `~/.cache` (may vary by distribution)
 - Windows: `{user}/AppData/Local`
- Expected content of the file (example):


```
{
  "access_token": "...",
  "client_id": "...",
  "client_secret": "...",
  "refresh_token": "...",
  ...
}
```



10.2 Secure Storage of Authentication Data

- Important: The authentication file content should be treated like a password and kept confidential.
- Challenge: How to use sensitive information in a public repository.
- Solution: Utilize GitHub *secrets* to store sensitive data securely.
- Action steps:
 - Navigate to the GitHub secrets option.
 - Create a secret named `GDRIVE_CREDENTIALS_DATA`.
 - Store the content of the authentication file found previously.
- Illustration: Example image provided showing GitHub secrets interface.



10.3 Workflow File Update and Validation

- Next step: Update the workflow file with the following code:


```

- uses: iterative/setup-dvc@v1
- name: Get data
  run: dvc pull
  env:
    GDRIVE_CREDENTIALS_DATA: ${
      secrets.GDRIVE_CREDENTIALS_DATA }

```
- Purpose: This code runs `dvc pull` using the secret authentication file.
- Additional resource: For guidance, visit the provided [small repository](#) that implements a similar workflow.
- Final steps:
 - Add changes to the repository.
 - Commit the updates.
 - Push the changes.
 - Confirm functionality: Ensure that unit tests relying on input data can run successfully.



Coding Practices Workflow Setup

- **Create a new workflow file** `codecheck.yaml`:
 - Set up the Python environment.
 - Install `ruff`.
 - Run `ruff check` and `ruff format` on the repository.
- **Add a step** to run `mypy` on the repository.
- **Ensure all steps pass**:
 - Focus on getting `ruff` to pass.
 - Optional: Attempt to get `mypy` passing, though it can be more challenging.





Knowledge check

- Four key concepts in GitHub Actions:
 - **Workflow:** A YAML file specifying instructions to run on certain events, located in the `.github/workflows` folder.
 - **Runner:** The environment where workflows execute, typically hosted by GitHub but can also be self-hosted.
 - **Job:** A collection of steps executed on the same runner; a workflow includes at least one job, often more.
 - **Action:** The smallest unit within a workflow; jobs consist of multiple actions executed sequentially.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

29

29



Knowledge check

- The `on` attribute specifies upon which events the workflow will be triggered. Assume you have set the `on` attribute to the following:


```
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  schedule:
    - cron: "0 0 * * *"
  workflow_dispatch: {}
```
- What 4 events would trigger the execution of that action?
 1. Direct push to branch `main` would trigger it
 2. Any pull request opened that will merge into `main` will trigger it
 3. At the end of the day the action would trigger, see [cron](#) for more info
 4. The trigger can be executed by manually triggering it through the GitHub UI, for example, shown below



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

30

30

